Upload CSV file

from google.colab import files
uploaded = files.upload()

Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.

```
Saving players_22.csv to players_22.csv
```

Cell 1 — Import Libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from pandas.plotting import scatter_matrix

Cell 2 — Load Dataset

```
df = pd.read_csv("players_22.csv", low_memory=False)
raw_df = df[["short_name", "age", "overall", "potential",
             "wage_eur", "height_cm", "weight_kg",
             "club_position", "nationality_name"]].copy()
raw_df.head()
```

	short_name	age	overall	potential	wage_eur	height_cm	weight_kg	club_position	nationality_name
0	L. Messi	34	93	93	320000.0	170	72	RW	Argentina
1	R. Lewandowski	32	92	92	270000.0	185	81	ST	Poland
2	Cristiano Ronaldo	36	91	91	270000.0	187	83	ST	Portugal
3	Neymar Jr	29	91	91	270000.0	175	68	LW	Brazil
4	K. De Bruyne	30	91	91	350000.0	181	70	RCM	Belgium

Cell 3 — First Look

```
print("Shape:", raw_df.shape)
raw_df.head()
```

```
Shape: (19239, 9)
```

	short_name	age	overall	potential	wage_eur	height_cm	weight_kg	club_position	nationality_name
0	L. Messi	34	93	93	320000.0	170	72	RW	Argentina
1	R. Lewandowski	32	92	92	270000.0	185	81	ST	Poland
2	Cristiano Ronaldo	36	91	91	270000.0	187	83	ST	Portugal
3	Neymar Jr	29	91	91	270000.0	175	68	LW	Brazil
4	K. De Bruyne	30	91	91	350000.0	181	70	RCM	Belgium

Cell 4 — Structure

```
raw_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19239 entries, 0 to 19238
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   short_name        19239 non-null  object 
 1   age               19239 non-null  int64  
 2   overall           19239 non-null  int64  
 3   potential         19239 non-null  int64  
 4   wage_eur          19178 non-null  float64
 5   height_cm         19239 non-null  int64  
 6   weight_kg         19239 non-null  int64  
 7   club_position     19178 non-null  object 
 8   nationality_name  19239 non-null  object 
dtypes: float64(1), int64(5), object(3)
memory usage: 1.3+ MB
```

Cell 5 — Variables Type

```
variable_types = pd.DataFrame({
    "Variable": raw_df.columns,
    "Suggested Type": ["ID", "Numerical", "Numerical", "Numerical",
                       "Numerical", "Numerical", "Numerical",
                       "Categorical", "Categorical"]
})
variable_types
```

	Variable	Suggested Type
0	short_name	ID
1	age	Numerical
2	overall	Numerical
3	potential	Numerical
4	wage_eur	Numerical
5	height_cm	Numerical
6	weight_kg	Numerical
7	club_position	Categorical
8	nationality_name	Categorical

Cell 6 — Missing Values

```
print("Missing values:")
raw_df.isnull().sum()
```

```
Missing values:
```

	0
short_name	0
age	0
overall	0
potential	0
wage_eur	61
height_cm	0
weight_kg	0
club_position	61
nationality_name	0

dtype: int64

Cell 7 — Duplicates

```
print("Duplicate rows:", raw_df.duplicated().sum())
```

```
Duplicate rows: 0
```

Cell 8 — Inconsistent Categories

```
print("Unique club_position values:")
print(raw_df["club_position"].unique())
```

```
Unique club_position values:
['RW' 'ST' 'LW' 'RCM' 'GK' 'CF' 'CDM' 'LCB' 'RDM' 'RS' 'LCM' 'SUB' 'CAM'
 'RCB' 'LDM' 'LB' 'RB' 'LM' 'RM' 'LS' 'CB' 'RES' nan 'RWB' 'RF' 'CM' 'LWB'
 'LAM' 'LF' 'RAM']
```

Cell 9 — Invalid Values

```
invalid_age = raw_df[(raw_df["age"] < 15) | (raw_df["age"] > 50)]
invalid_overall = raw_df[(raw_df["overall"] < 0) | (raw_df["overall"] >
100)]

print("Invalid age rows:")
display(invalid_age)
print("Invalid overall rows:")
display(invalid_overall)
```

```
Invalid age rows:
```

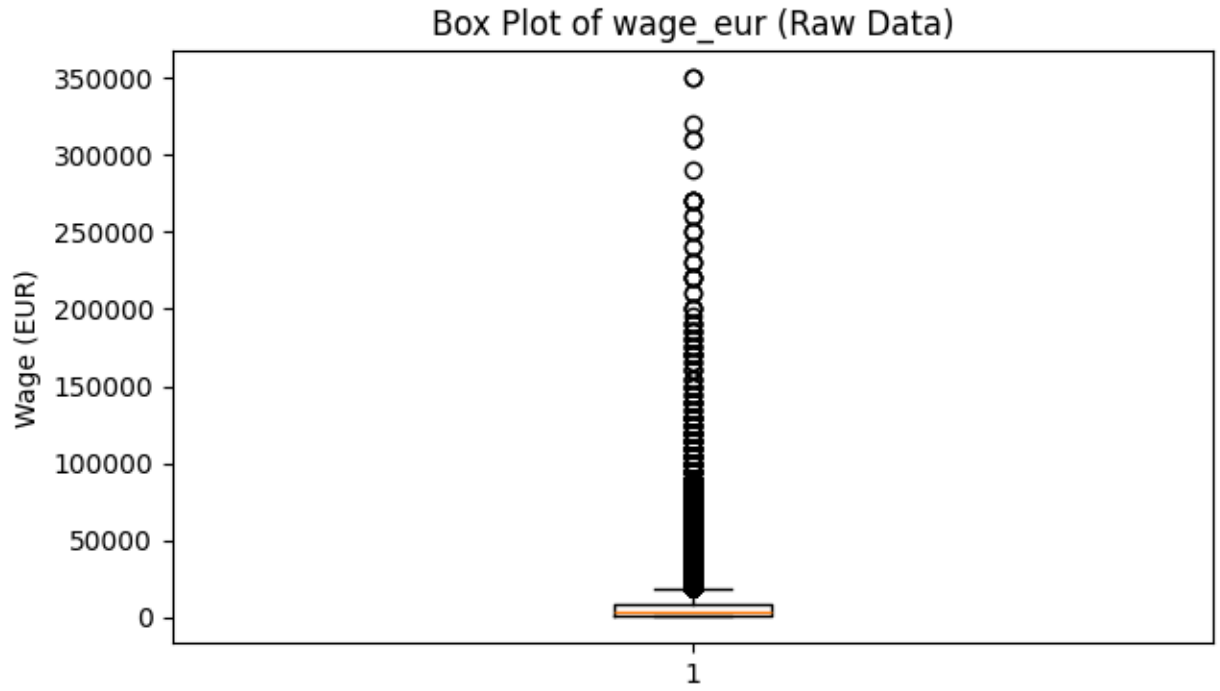
	short_name	age	overall	potential	wage_eur	height_cm	weight_kg	club_position	nationality_name
16209	K. Miura	54	59	59	700.0	177	72	RES	Japan

```
Invalid overall rows:
```

	short_name	age	overall	potential	wage_eur	height_cm	weight_kg	club_position	nationality_name

Cell 10 — Outlier Check

```
plt.figure(figsize=(7, 4))
plt.boxplot(raw_df["wage_eur"].dropna())
plt.title("Box Plot of wage_eur (Raw Data)")
plt.ylabel("Wage (EUR)")
plt.show()
```



Cell 11 — Clean the Data

```
clean_df = raw_df.copy()
# Standardize club_position labels
clean_df["club_position"] =
clean_df["club_position"].str.strip().str.upper()
# Fill missing wage with median
median_wage = clean_df["wage_eur"].median()
clean_df["wage_eur"] = clean_df["wage_eur"].fillna(median_wage)
# Fill missing height and weight with median
clean_df["height_cm"] =
clean_df["height_cm"].fillna(clean_df["height_cm"].median())
clean_df["weight_kg"] =
clean_df["weight_kg"].fillna(clean_df["weight_kg"].median())
# Drop rows where club_position is still missing
clean_df = clean_df.dropna(subset=["club_position"])
# Remove duplicates
clean_df = clean_df.drop_duplicates()
print("Median wage used:", median_wage)
print("Shape after cleaning:", clean_df.shape)
```

```
Median wage used: 3000.0
Shape after cleaning: (19178, 9)
```

Cell 12 — Verify Cleaning

```
print("Missing values after cleaning:")
display(clean_df.isnull().sum())
print("Duplicates after cleaning:", clean_df.duplicated().sum())
print("Unique club_position values:")
print(clean_df["club_position"].unique())
```

```
Missing values after cleaning:
```

	0
short_name	0
age	0
overall	0
potential	0
wage_eur	0
height_cm	0
weight_kg	0
club_position	0
nationality_name	0

dtype: int64

```
Duplicates after cleaning: 0
Unique club_position values:
['RW' 'ST' 'LW' 'RCM' 'GK' 'CF' 'CDM' 'LCB' 'RDM' 'RS' 'LCM' 'SUB' 'CAM'
 'RCB' 'LDM' 'LB' 'RB' 'LM' 'RM' 'LS' 'CB' 'RES' 'RWB' 'RF' 'CM' 'LWB'
 'LAM' 'LF' 'RAM']
```

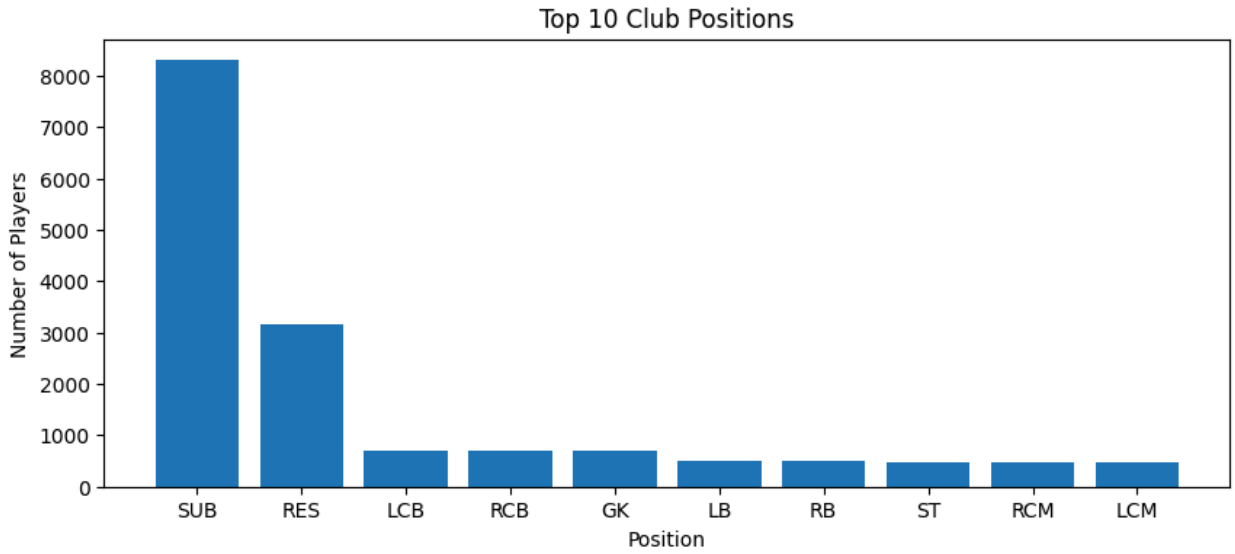
Cell 13 — Univariate EDA: Categorical (club_position)

```
pos_counts = clean_df["club_position"].value_counts().head(10)
pos_percent =
clean_df["club_position"].value_counts(normalize=True).head(10) * 100

pos_summary = pd.DataFrame({
    "Frequency": pos_counts,
    "Percentage": pos_percent.round(2)
})
print(pos_summary)
```

```
plt.figure(figsize=(10, 4))
plt.bar(pos_counts.index, pos_counts.values)
plt.title("Top 10 Club Positions")
plt.xlabel("Position")
plt.ylabel("Number of Players")
plt.show()
```

```
               Frequency  Percentage
club_position
SUB                 8299       43.27
RES                 3168       16.52
LCB                  701        3.66
RCB                  701        3.66
GK                   701        3.66
LB                   515        2.69
RB                   515        2.69
ST                   476        2.48
RCM                  470        2.45
LCM                  470        2.45
```



Cell 14 — Univariate EDA: Numerical (overall)

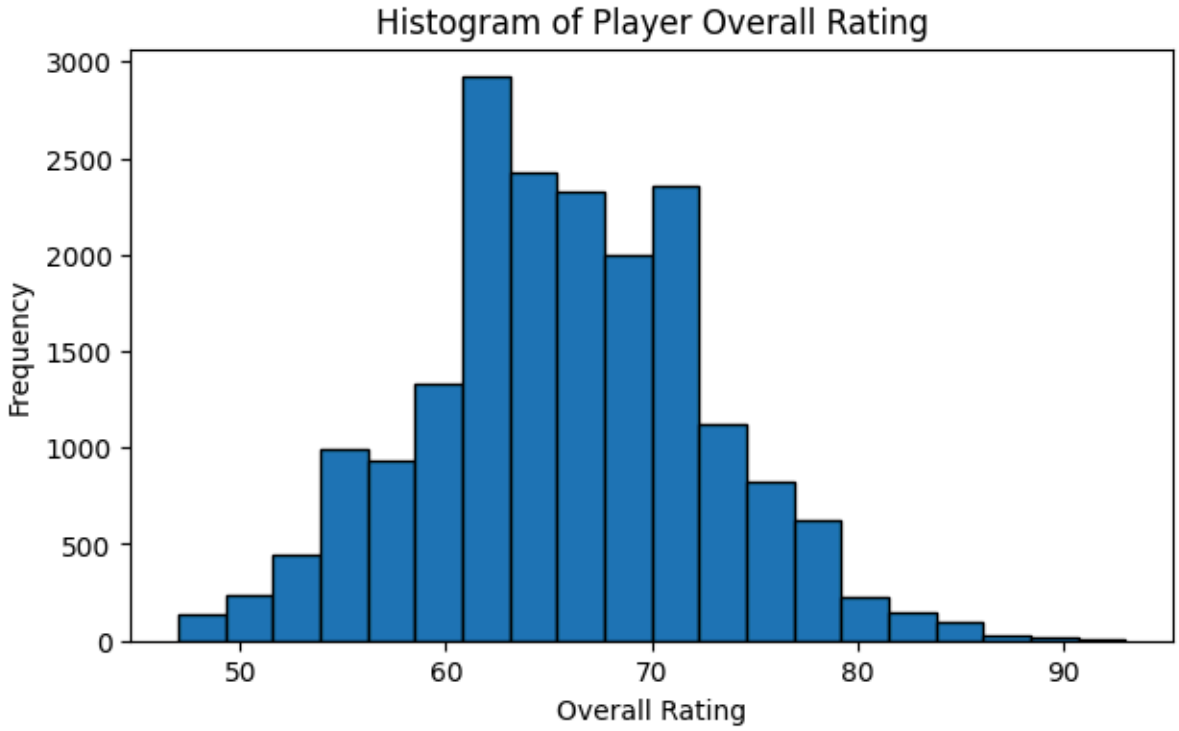
```
print(clean_df["overall"].describe())
```

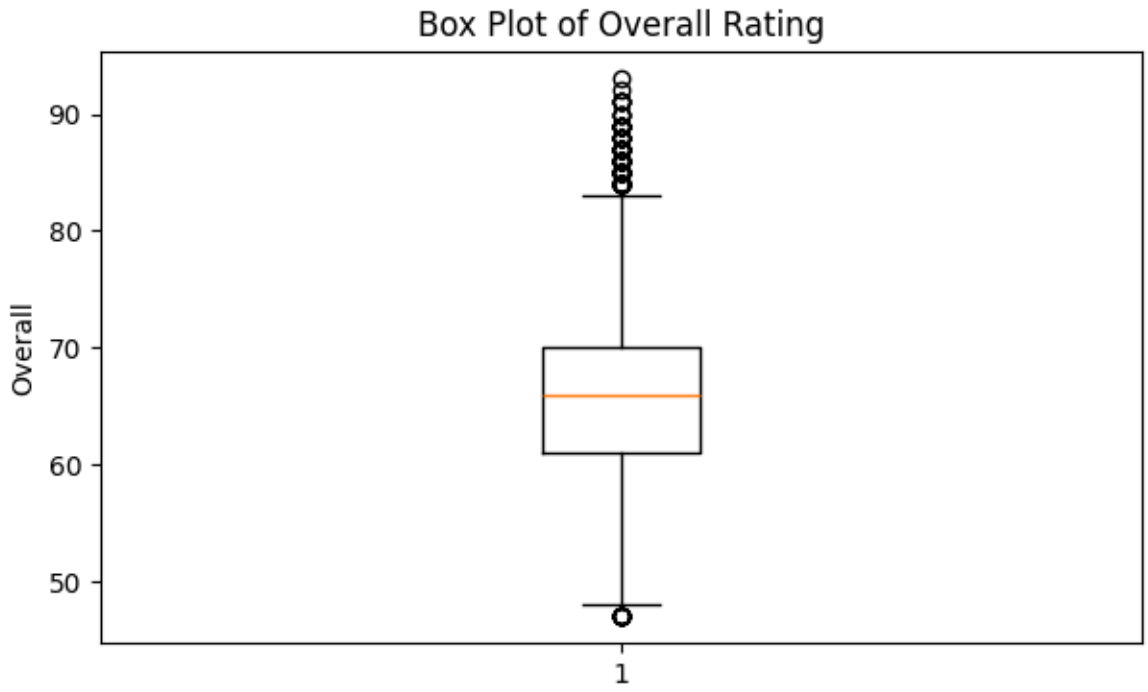
```
plt.figure(figsize=(7, 4))
plt.hist(clean_df["overall"], bins=20, edgecolor="black")
plt.title("Histogram of Player Overall Rating")
plt.xlabel("Overall Rating")
plt.ylabel("Frequency")
```

```
plt.show()

plt.figure(figsize=(7, 4))
plt.boxplot(clean_df["overall"])
plt.title("Box Plot of Overall Rating")
plt.ylabel("Overall")
plt.show()
```

```
count    19178.000000
mean        65.760246
std          6.882432
min         47.000000
25%         61.000000
50%         66.000000
75%         70.000000
max         93.000000
Name: overall, dtype: float64
```

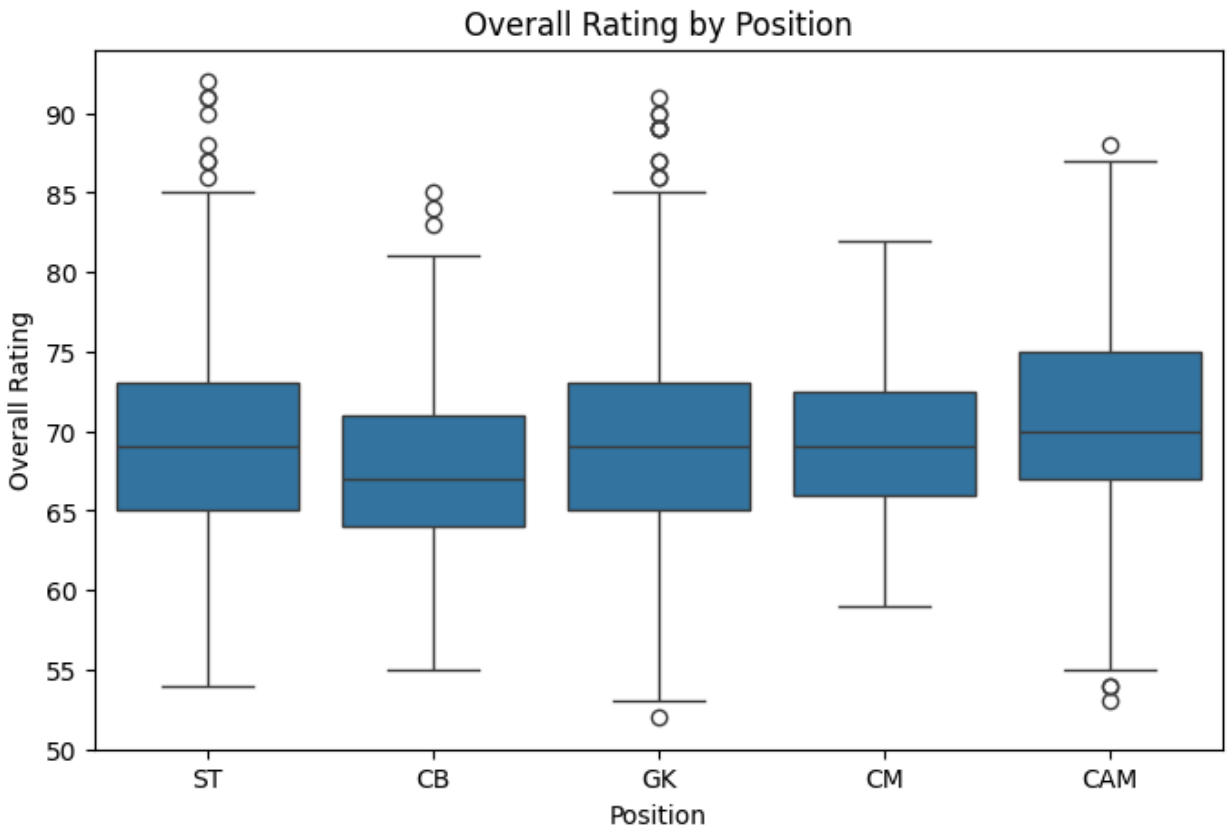




Cell 15 —EDA

```
top_positions = ["ST", "CB", "GK", "CM", "CAM"]
filtered = clean_df[clean_df["club_position"].isin(top_positions)]

plt.figure(figsize=(8, 5))
sns.boxplot(data=filtered, x="club_position", y="overall",
order=top_positions)
plt.title("Overall Rating by Position")
plt.xlabel("Position")
plt.ylabel("Overall Rating")
plt.show()
```



Cell 16—Bivariate EDA: Categorical vs Categorical

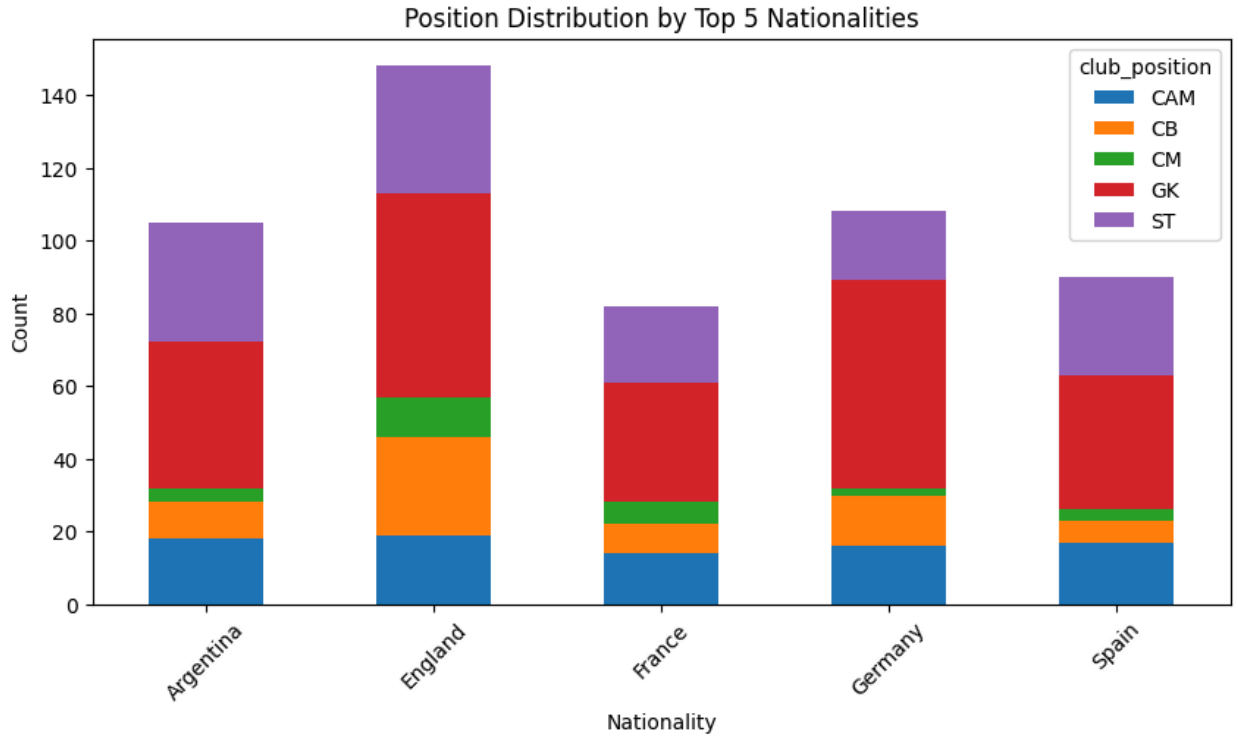
```
top_nations = clean_df["nationality_name"].value_counts().head(5).index

filtered_nat = clean_df[clean_df["nationality_name"].isin(top_nations)]
filtered_nat =
filtered_nat[filtered_nat["club_position"].isin(top_positions)]

crosstab = pd.crosstab(filtered_nat["nationality_name"],
filtered_nat["club_position"])
print(crosstab)

crosstab.plot(kind="bar", stacked=True, figsize=(10, 5))
plt.title("Position Distribution by Top 5 Nationalities")
plt.xlabel("Nationality")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```

club_position	CAM	CB	CM	GK	ST
nationality_name					
Argentina	18	10	4	40	33
England	19	27	11	56	35
France	14	8	6	33	21
Germany	16	14	2	57	19
Spain	17	6	3	37	27



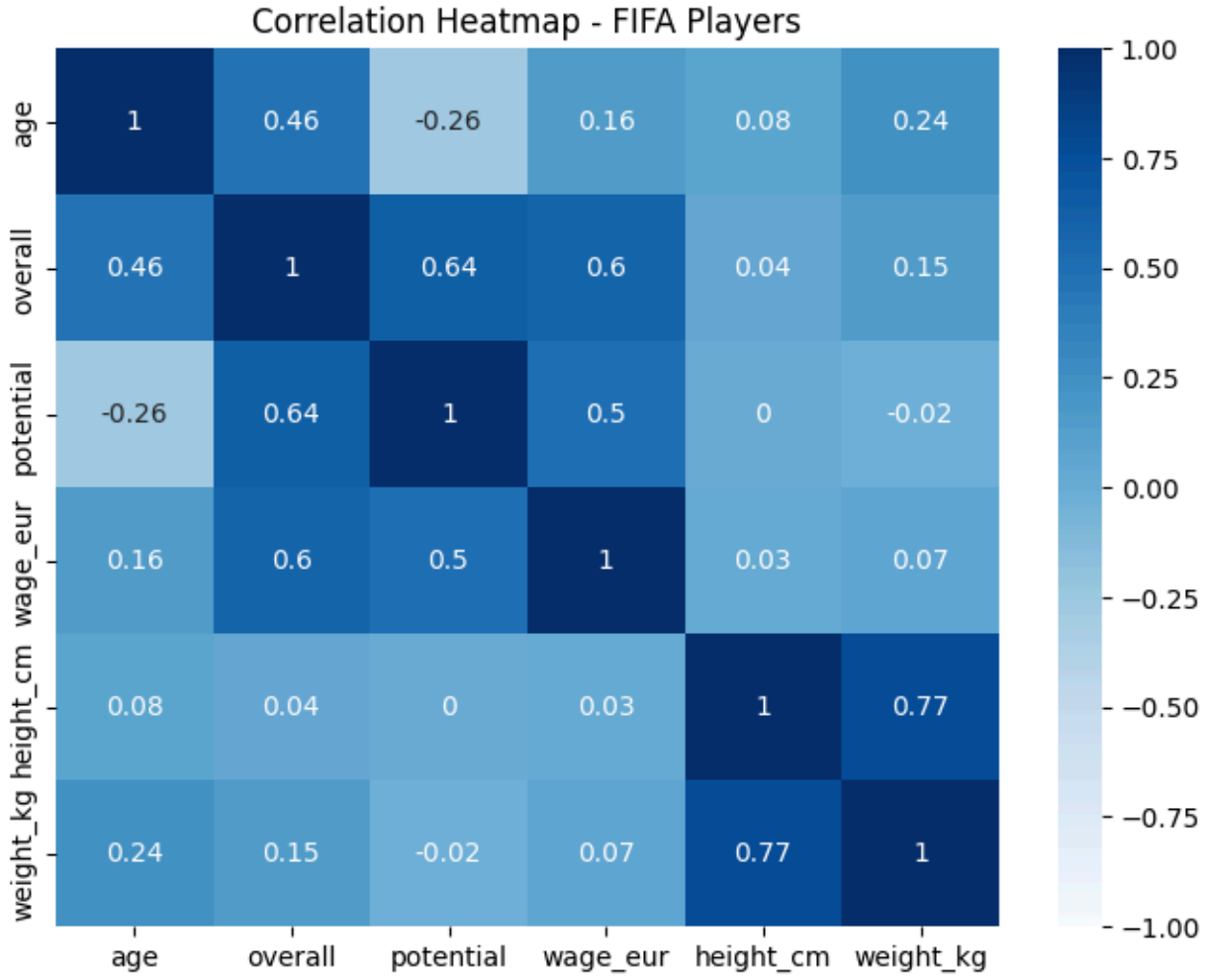
Cell 17 —Correlation Heatmap

```
numeric_cols = ["age", "overall", "potential", "wage_eur", "height_cm",
"weight_kg"]
corr_matrix = clean_df[numeric_cols].corr().round(2)
print(corr_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="Blues", vmin=-1, vmax=1)
plt.title("Correlation Heatmap - FIFA Players")
plt.show()
```

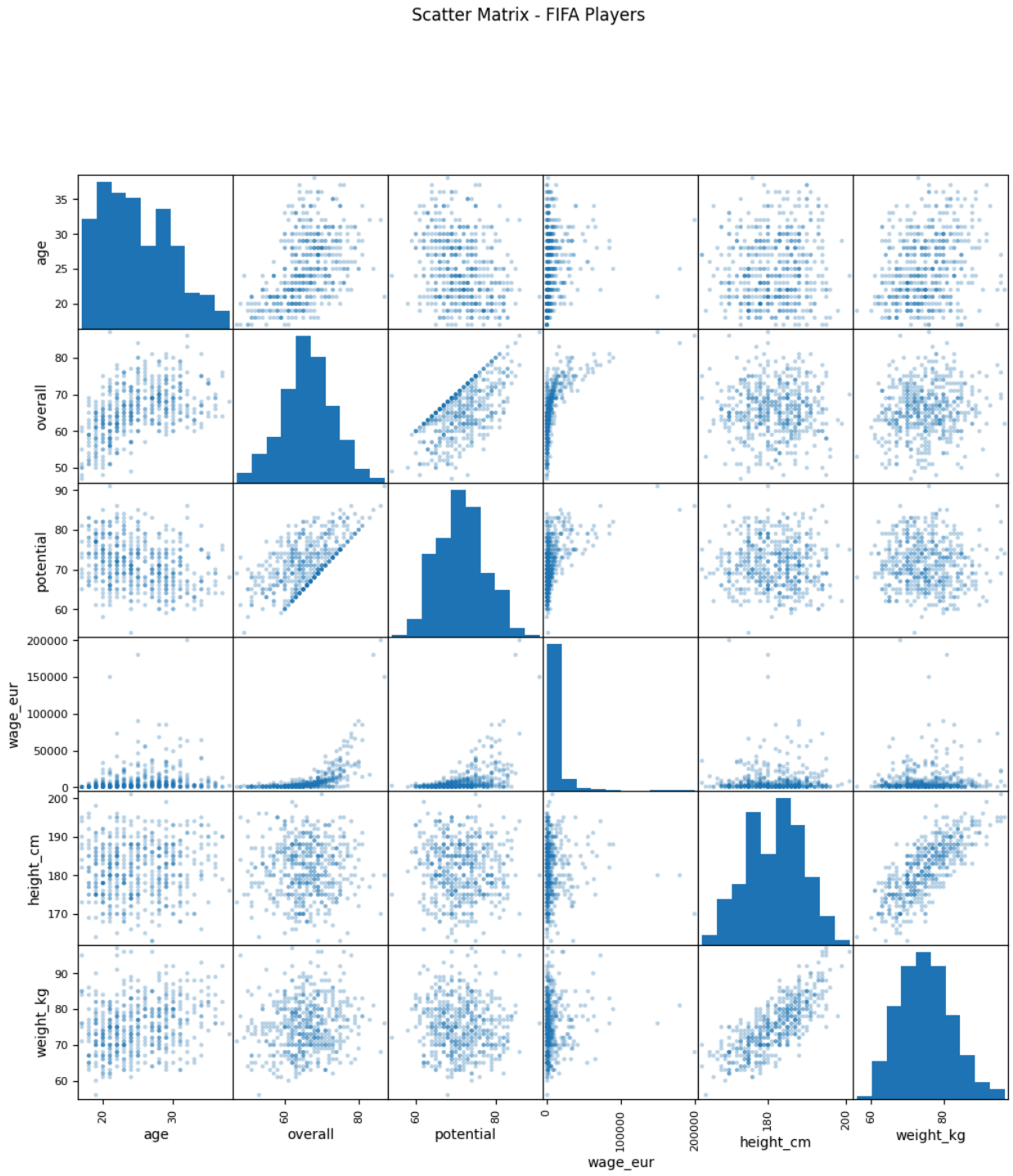
	age	overall	potential	wage_eur	height_cm	weight_kg
age	1.00	0.46	-0.26	0.16	0.08	0.24
overall	0.46	1.00	0.64	0.60	0.04	0.15
potential	-0.26	0.64	1.00	0.50	0.00	-0.02

```
wage_eur     0.16       0.60         0.50         1.00         0.03         0.07
height_cm    0.08       0.04         0.00         0.03         1.00         0.77
weight_kg    0.24       0.15        -0.02         0.07         0.77         1.00
```



Cell 18 — Pair Plot

```
numeric_cols = ["age", "overall", "potential", "wage_eur", "height_cm",
"weight_kg"]
scatter_matrix(clean_df[numeric_cols].sample(500), figsize=(12, 12),
diagonal="hist", alpha=0.3)
plt.suptitle("Scatter Matrix - FIFA Players", y=1.02)
plt.show()
```



Cell 19 — EDA Findings

```
print("FIFA Players EDA Findings:")
print("1. Dataset contains", clean_df.shape[0], "players and",
clean_df.shape[1], "variables.")
print("2. Most players are rated between 60 and 75 overall.")
```

```
print("3. Player ratings peak around ages 27-30 then slightly decline.")
print("4. Wage has strong outliers — top stars earn far more than average players.")
print("5. Overall rating and potential are strongly positively correlated.")
```

```
FIFA Players EDA Findings:
1. Dataset contains 19178 players and 9 variables.
2. Most players are rated between 60 and 75 overall.
3. Player ratings peak around ages 27-30 then slightly decline.
4. Wage has strong outliers — top stars earn far more than average players.
5. Overall rating and potential are strongly positively correlated.
```
